Handwritten English Alphabet Recognition Using Convolutional Neural Networks

A CNN-based image classification project on the A–Z Handwritten Alphabets dataset

Project Proposal

Submitted by:
Wai Yan Tun
MSc Data Science
University of Europe for Applied Sciences

Supervisor: Prof. Dr. Raja Hashim Ali

Date: 07.06.2026

Table of Contents

1. Project Title

Handwritten English Alphabet Recognition Using Convolutional Neural Networks.

2. Background and Motivation

Reading handwritten text is something people can do easily but for computer, it's not that easy. Handwriting are different between different people, and even the same person writes differently sometime. Therefore, teaching a computer to recognize letters reliably is a useful and well-studied problem. A very familiar example is a doctor's handwriting: prescriptions and medical notes are often so hard to read that normal people cannot understand them, and even pharmacists have to guess sometimes what is written. Misreading such notes can lead to serious mistakes. A system that can turn this kind of messy human handwriting into clear, readable text would be a lot of help to ordinary people. This is the main goal of Optical Character Recognition (OCR), which converts scanned or photographed text into editable digital text.

Other than medical notes, handwriting recognition has many other real uses. It can be used to read the postal addresses, the digitizing of old paper and medical records, the processing of filled-in forms, and assistive tools for people with visual or motor difficulties. Most modern OCR systems work by breaking a word down into single characters and recognizing each one in turn, so a reliable single-letter classifier is the basic building block on which these larger systems are built. This project focuses on that building block: recognizing one handwritten capital letter at a time, which is the first step toward reading harder handwriting such as a doctor's notes.

Convolutional Neural Networks (CNNs) are good tool for this kind of project. Unlike older methods that needed hand-designed features, CNN learns useful patterns (edges, curves, strokes) straight from the pixels. This project uses a CNN because the input is small, fixed-size image data, which is exactly what CNNs handle well. It also gives a good chance to practice the full deep-learning workflow on a manageable but realistic dataset.

3. Problem Statement

Handwritten character recognition is an important study of computer vision and machine learning. They have been widely using in many cases such as document digitization, form processing, and automated data entry. This project is a 26-class image classification problem. The model must identify the correct class label from A to Z from the given 28×28 grayscale image of a handwritten capital letter. Although Convolutional Neural Networks (CNNs) have achieved the strong performance in image recognition, identifying the handwritten characters is still challenging due to the variations of different writing style and character shapes.

There are two key difficulties which is making this project harder than it first looks. First, several letters look alike when they are written by hand such as I and L, O and Q, or U and V. So, model can confuse them. Second, the dataset is imbalanced. Some letters appear ten of thousand of times while others appear only about a thousand times. Because of that model can reach high overall accuracy while still doing poorly on the rare letters. Therefore, the project must use both overall and per class metrics to ensure reliable classification.

4. Selected Dataset and Dataset Link

The project uses the A–Z Handwritten Alphabets in CSV format dataset, published on Kaggle by Sachin Patel. The dataset is derived from the NIST handwritten character database and a related letter set. Each handwritten letter has been cleaned, centred, and resized to a 28×28 grayscale image, then flattened into a single row in the CSV. This is the same format made by the MNIST digit dataset, but for the 26 capital letters instead of digits.

Dataset link: https://www.kaggle.com/datasets/sachinpatel21/az-handwritten-alphabets-in-csv-format

5. Dataset Description

The whole dataset is stored in one CSV file. Each row is one image. The first column is the class label (an integer from 0 to 25, where 0 = A and 25 = Z). The remaining 784 columns are the pixel values of a 28×28 image, each in the range 0–255. The file has 785 columns in total and no header row.

5.1 Summary

Property	Value
Total images	372,451 rows (one image per row)
Number of classes	26 (capital letters A–Z)
Image type	Grayscale, single channel
Image size	28 × 28 pixels (784 values, flattened)
Pixel range	0–255 (0 = background, higher = ink)
File format	Single CSV file, 785 columns, no header
Label encoding	Column 1 = integer 0–25 (0 = A … 25 = Z)
Class balance	Heavily imbalanced (see Figure 1)

Table 1. Summary of the A–Z Handwritten Alphabets dataset.

5.2 Class distribution

The dataset is a very imbalanced dataset. The most common letter, O, has 57,825 images, while the least common, I, has only 1,120. That is roughly a 50 times gap. The letters O, S, U, T, and C dominate, while F, I, V, and K are rare. Figure 1 shows the full distribution, and Table 2 lists the exact counts.

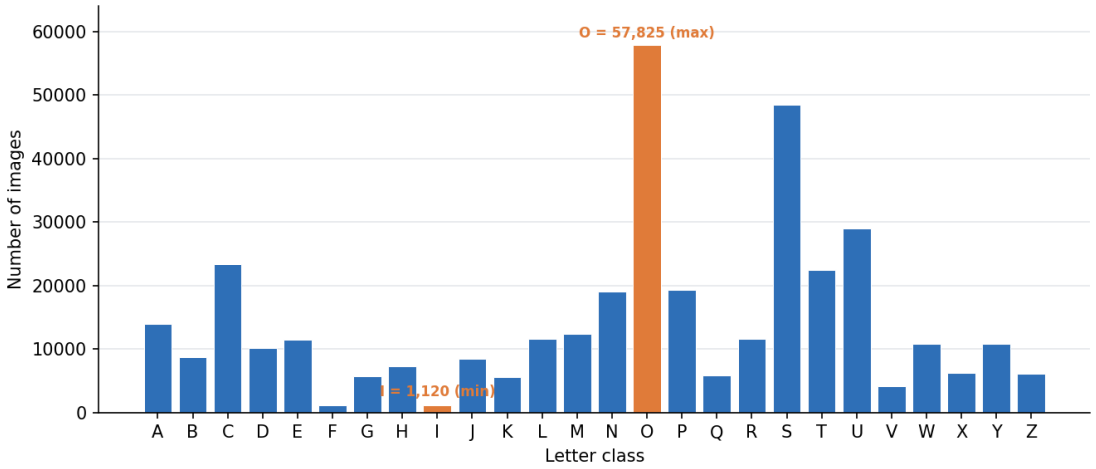

Figure 1. Number of images per letter, computed from the full dataset.

Letter	Count	Letter	Count	Letter	Count
A	13,870	J	8,493	S	48,419
B	8,668	K	5,603	T	22,495
C	23,409	L	11,586	U	29,008
D	10,134	M	12,336	V	4,182
E	11,440	N	19,010	W	10,784
F	1,163	O	57,825	X	6,272
G	5,762	P	19,341	Y	10,859
H	7,218	Q	5,812	Z	6,076
I	1,120	R	11,566	—	—

Table 2. Exact per-class image counts. Total = 372,451 images.

5.3 Sample images

Figure 2 shows one real example of each letter, reshaped from the CSV back into a 28×28 image. The samples confirm the data is clean and centered, and they show the natural variety in handwriting that the model must deal with.

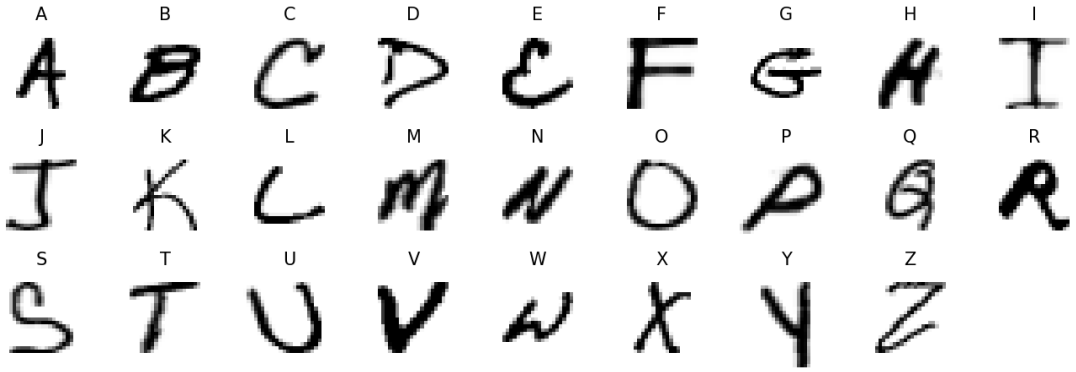

Figure 2. One actual sample image per class, drawn directly from the dataset.

6. Research Questions

The project will be guided by the following research questions:

RQ1. How accurately can a custom CNN classify the 26 handwritten letters on a held-out test set?

RQ2. Which letters are most often confused with each other, and do these match the look-alike pairs we expect (for example I/L, O/Q, U/V)?

RQ3. How does the heavy class imbalance affect performance on rare letters, and does data augmentation reduce this gap?

RQ4. Does a transfer-learning model (a pre-trained network fine-tuned on this data) beat a CNN trained from scratch, given that the images are small and grayscale?

RQ5. Which input regions does the model rely on when it makes a decision, as revealed by Grad-CAM?

7. Expected Results

This section gives the expected outcome for each research question, with the supporting figure or table for each. Table 3 summarizes all five expected answers, and the subsections that follow explain each in turn.

RQ	Research question (short)	Expected finding
RQ1	Overall accuracy of the custom CNN	High overall test accuracy, around 97–99%
RQ2	Which letters are confused	Errors cluster at look-alike pairs (I/L, O/Q, U/V)
RQ3	Effect of class imbalance	Rare letters (I, F, V) get lower recall; macro F1 < weighted F1
RQ4	Transfer learning vs CNN	Custom CNN matches or slightly beats MobileNetV2

RQ	Research question (short)	Expected finding
RQ5	What the model looks at	Grad-CAM focuses on the ink strokes of each letter

Table 3. Expected results for each research question.

7.1 RQ1 – Overall accuracy

Based on published results of MNIST-style letter recognition, a well-trained CNN should reach roughly 97–99% overall test accuracy on this dataset. However, because the data is imbalanced, the macro-averaged F1 is expected to be a few points lower than the overall accuracy. The gap between weighted and macro F1 is itself a key result, since it measures how much the rare letters are being neglected. The training curves should rise quickly in the first few epochs and then flatten, with validation accuracy tracking training accuracy closely if augmentation and dropout control overfitting.

Metric	Expected value	What it tells us
Test accuracy	~0.98	Overall fraction of letters classified correctly
Macro F1	~0.95	Average F1 across all 26 letters, equally weighted
Weighted F1	~0.98	F1 weighted by class size (dominated by common letters)
Worst-class recall	~0.85	Recall on the hardest rare letter (e.g. I or F)
Top-3 accuracy	~0.997	True letter is within the model's top 3 guesses

Table 4. Expected evaluation metrics.

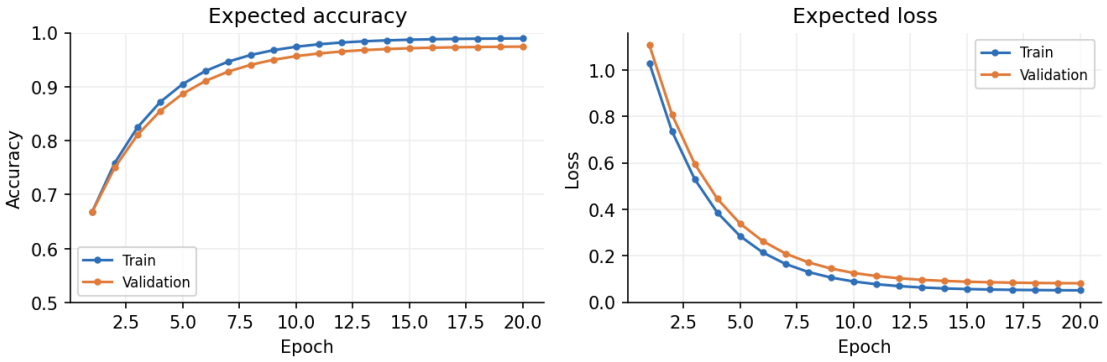

Figure 3. Expected accuracy and loss curves.

7.2 RQ2 – Which letters are confused

The confusion matrix is expected to show a strong diagonal (most letters classified correctly) with small off-diagonal clusters at the letter pairs that look alike when written by hand. The most

likely confusions are I with L, O with Q, U with V, and E with F. These are the same pairs a human would also hesitate, that's why seeing them here would confirm the model is making sensible mistakes rather than random ones.

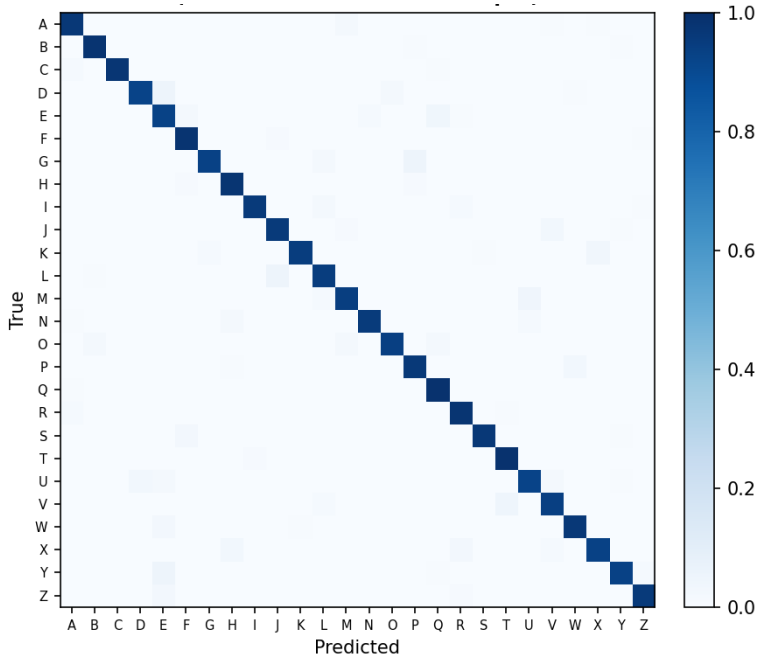

Figure 4. Expected normalized confusion matrix.

7.3 RQ3 – Effect of class imbalance

Since some letters appear more less often than others, the model is expected to do worse on the rare ones. Figure 5 shows the expected recall for each letter: the common letters sit near the top, while the rare letters (such as I, F, and V) fall noticeably below the average line. Data augmentation, and optionally class weights, are expected to lift the rare-letter recall and narrow this gap, though probably not close it completely.

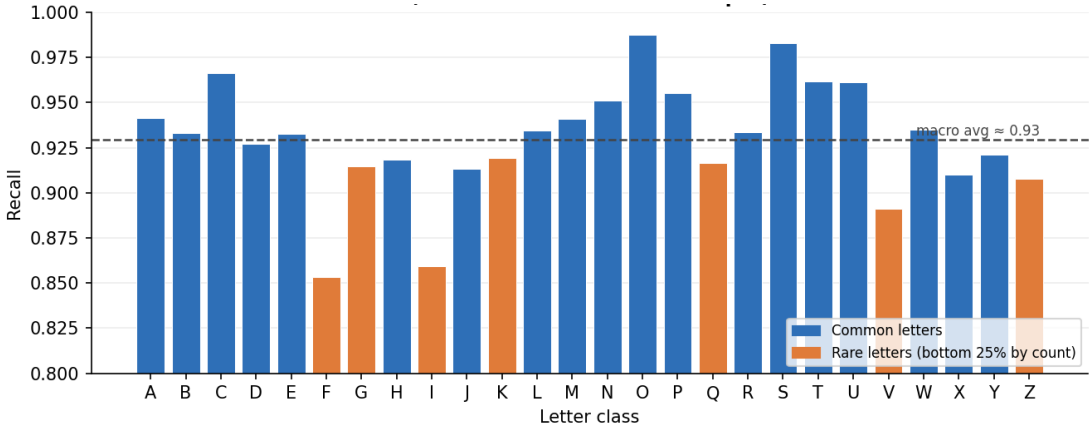

Figure 5. Expected per-class recall, with rare letters lagging.

7.4 RQ4 – Transfer learning vs custom CNN

Transfer-learning models were trained on large color photographs, so their pre-learned features may not help much on tiny grayscale letters. The expectation is that the custom CNN will match or slightly better than MobileNetV2, while the transfer model may train in fewer epochs. The comparison is reported on the same test set using accuracy, macro F1, and weighted F1, shown in Figure 6 and Table 5.

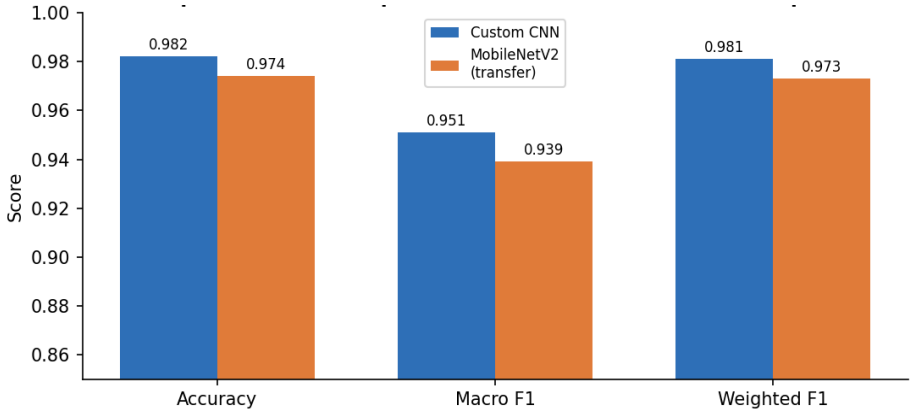

Figure 6. Expected model comparison.

Model	Accuracy	Macro F1	Weighted F1
Custom CNN	~0.982	~0.951	~0.981
MobileNetV2 (transfer)	~0.974	~0.939	~0.973

Table 5. Expected final model comparison.

7.5 RQ5 – What the model looks at

Grad-CAM highlights the parts of the image that most influenced the prediction. The expectation is that the heat will sit on the actual ink that define each letter (for example the crossbar of an A or the tail of a Q), and not on the empty background. If the heat lands on the strokes, it is good evidence that the model learned meaningful shapes rather than noise.

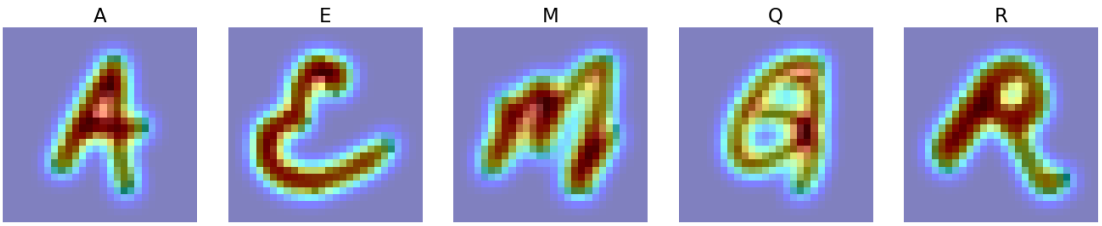

Figure 7. Expected Grad-CAM overlays on sample letters.

8. Proposed Methodology

The project follows a standard, reproducible deep-learning pipeline. Each step below is implemented as a clear section in the Kaggle notebook.

1. Load the CSV into a DataFrame and split it into pixel features and labels.

2. Reshape each 784-value row into a 28×28×1 image and normalize pixels to the 0–1 range.
3. Split the data into training, validation, and test sets (about 70/15/15), using stratified sampling so every letter keeps the same proportion in each split.
4. Apply data augmentation (small rotations, shifts, and zoom) to the training set to add variety and help rare classes.
5. Build and train a custom CNN with the architecture in Section 9.
6. Build a transfer-learning model (Section 10) and train it on the same splits for a fair comparison.
7. Evaluate both models on the untouched test set using the metrics in Section 11.
8. Produce the confusion matrix, classification report, accuracy/loss curves, model-comparison table, and Grad-CAM visualizations.
9. Save the trained model, the key figures, and the metrics so the results are reproducible.

To handle the class imbalance, the project will use stratified splitting and augmentation, and may also apply class weights during training so rare letters are not ignored. All randomness is seeded so the run can be repeated.

9. CNN Model Design

The custom CNN is a compact stack of convolution and pooling blocks followed by dense layers. It is very small because the images are only 28×28. So, a heavy network would overfit and waste compute. Figure 8 shows the overall structure and Table 6 lists the layers.

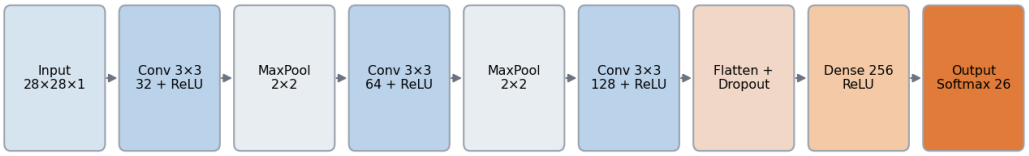

Figure 8. Proposed CNN architecture (conceptual).

Layer	Configuration	Activation	Output
Input	28×28×1 grayscale	—	28×28×1
Conv2D	32 filters, 3×3, padding same	ReLU	28×28×32
MaxPooling2D	2×2	—	14×14×32
Conv2D	64 filters, 3×3, padding same	ReLU	14×14×64
MaxPooling2D	2×2	—	7×7×64
Conv2D	128 filters, 3×3, padding same	ReLU	7×7×128

Layer	Configuration	Activation	Output
MaxPooling2D	2×2	—	3×3×128
Flatten + Dropout	dropout 0.4	—	1152
Dense	256 units	ReLU	256
Dense (output)	26 units	Softmax	26

Table 6. Layer-by-layer design of the proposed CNN.

Hyperparameter	Planned setting
Loss function	Categorical (or sparse categorical) cross-entropy
Optimizer	Adam, learning rate 0.001
Batch size	128
Epochs	Up to 20, with early stopping on validation loss
Regularization	Dropout 0.4, data augmentation, optional class weights
Callbacks	EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

Table 7. Planned training hyperparameters.

10. Transfer Learning Model

To compare with the custom CNN, the project uses a pre-trained model, MobileNetV2 (trained on ImageNet). MobileNetV2 is chosen because it is small and fast, which suits a notebook environment. The point of this comparison is research question 4: “Does a transfer-learning model (a pre-trained network fine-tuned on this data) beat a CNN trained from scratch, given that the images are small and grayscale?”

Pre-trained networks expect larger and color images but this dataset is small and grayscale. Therefore, each 28×28 image is resized up (for example to 32×32 or 96×96) and the single channel is repeated three times to make a pseudo-RGB image. At first, the base model's convolutional layers are frozen and used as a feature extractor; a new classification head with a 26-way SoftMax is trained on top. An optional second stage unfreezes the top layers for fine-tuning. Both models are trained and tested on the same splits so the comparison is fair.

11. Evaluation Metrics

Since the data is imbalanced, overall accuracy alone is not enough to evaluate the model. Therefore, other metrics are used to make sure that rare letters are not overlooked.

- Accuracy: the overall fraction of correctly classified letters.
- Precision, recall, and F1-score per class: how the model does on each individual letter.
- Macro-averaged F1: the average F1 across all 26 letters, treating each letter equally regardless of size.
- Weighted-average F1: F1 weighted by class size, which reflects the common letters more.
- Confusion matrix: shows exactly which letters are confused with which.

- Accuracy and loss curves: track training and validation performance across epochs to check for overfitting.
- ROC curves and AUC (one-vs-rest): an optional view of class separability.

The comparison between the custom CNN and the transfer-learning model will be reported using accuracy, macro F1, and weighted F1 together.

12. Expected Figures and Tables

- Figure 1 – Class distribution bar chart.
- Figure 2 – Sample images grid.
- Figure 3 – Accuracy and loss curves.
- Figure 4 – Confusion matrix.
- Figure 5 – Per-class recall vs class size.
- Figure 6 – Model comparison chart, CNN vs transfer learning.
- Figure 7 – Grad-CAM visualizations for sample predictions.
- Figure 8 – CNN architecture diagram.
- Table 1 – Dataset summary.
- Table 2 – Per-class counts.
- Table 3 – Expected results per research question.
- Table 4 – Expected evaluation metrics.
- Table 5 – Model comparison, CNN vs transfer learning.
- Table 6 – CNN layer design.
- Table 7 – Training hyperparameters.

Repository

Source code, notebook, README, dataset link, and output figures will be organized in the public GitHub repository:
https://github.com/Waiyan172/Mechine-Learning-Project-UE